Synthesizable Verilog

Verilog includes many constructs for describing and simulating digital systems, but not every simulation construct describes hardware that a synthesis tool can build. The synthesizable subset depends in part on the target technology and tool, so check the tool's documentation when portability matters. The examples below focus on combinational logic.

1 Synthesis Rules for Combinational Logic

The output of a combinational circuit at time t should depend only on the inputs at time t; it should not depend on stored state or on the circuit's past history. Follow these rules when describing combinational logic:

- Describe the required functionality, not technology-specific timing.
- Avoid feedback paths from outputs back to inputs.
- Assign every output for every possible input case. In an incomplete `if` or `case`, a tool may infer a latch to retain the previous value when no branch assigns the output.

Combinational logic can be described with gate primitives, continuous assignments, procedural statements, functions, and hierarchy. Combinational user-defined primitives (UDPs) are another option, but support varies among synthesis tools. Tasks can also be used when their contents avoid simulation timing controls.

2 Combinational Coding Styles

2.1 Gate-level netlist

Built-in gate primitives make the logic connections explicit. A synthesis tool maps the described logic to gates in the target library and may optimize the netlist.

```
module example (x1, x2, x3, x4, y);
  input x1, x2, x3, x4;
  output y;
  wire w1, w2, w3;

  or   (w1, x1, x2);
  or   (w2, x3, x4);
  xor  (w3, x3, x4);
  nand (y, w1, w2, w3);
endmodule
```

2.2 Continuous assignment

An `assign` statement continuously drives a net from a Boolean expression. The synthesis tool maps and optimizes the expression for the target technology.

```
module carry (cout, a, b, c);
  input a, b, c;
  output cout;

  assign cout = (a & b) | (b & c) | (c & a);
endmodule
```

2.3 Procedural blocking assignment

An `always` block can describe combinational behavior. Include every signal read by the block in its sensitivity list; otherwise simulation may not reevaluate the block when an input changes. The complete assignments in this multiplexer avoid inferring a latch.

```
module mux2to1 (f, in0, in1, sel);
  input in0, in1, sel;
  output reg f;

  always @(in0 or in1 or sel)
    if (sel) f = in1;
    else     f = in0;
endmodule
```

The blocking assignment operator (`=`) updates the procedural value immediately and is commonly used for combinational logic. In newer Verilog code, `always @*` can automatically include the signals read by the block in the sensitivity list.

2.4 Functions

A function returns one value, executes without advancing simulation time, and cannot contain delay or event controls. Functions can help reuse and organize combinational expressions. In the example, each function returns its result by assigning to its own name.

```
module fulladder (s, cout, a, b, cin);
  input a, b, cin;
  output s, cout;

  assign s    = sum(a, b, cin);
  assign cout = carry(a, b, cin);

  function sum;
    input x, y, z;
    begin
      sum = x ^ y ^ z;
    end
  endfunction

  function carry;
    input x, y, z;
    begin
      carry = (x & y) | (y & z) | (z & x);
    end
  endfunction
endmodule
```

2.5 Tasks

A task can return multiple values through output arguments. Its arguments are passed in the order declared in the task. A task used for combinational synthesis must not contain simulation delays or event controls. The delay shown in the source image (`#2`) is intentionally omitted here because it describes simulation timing, not combinational hardware.

```
module fulladder (s, cout, a, b, cin);
  input a, b, cin;
  output reg s, cout;

  always @(a or b or cin)
    FA (s, cout, a, b, cin);

  task FA;
    output sum, carry;
    input A, B, C;
    begin
      sum   = A ^ B ^ C;
      carry = (A & B) | (B & C) | (C & A);
    end
  endtask
endmodule
```

2.6 Function and task differences

- A function returns one value; a task can provide multiple values through `output` or `inout` arguments.
- A function executes in zero simulation time and cannot contain delay, event, or timing-control statements. A task may contain such controls for simulation, but those controls generally make it unsuitable for combinational synthesis.
- A function can call another function, but not a task. A task can call functions and other tasks.
- A task may have input, output, and inout arguments. Traditional Verilog functions take input arguments and return their result through the function name.

3 Constructs to Avoid in Combinational Synthesis

The following constructs either describe behavior other than combinational logic or are not consistently supported for synthesis:

- Edge-dependent event controls, such as `@(posedge clk)`, describe sequential logic rather than combinational logic.
- Combinational feedback loops do not describe an ordinary acyclic combinational circuit.
- Delays and event controls inside procedural or continuous assignments are simulation constructs, not hardware timing specifications.
- Procedural loops with timing controls cannot be implemented as ordinary combinational logic. Loops should have bounds that synthesis can determine; a data-dependent or unbounded loop may not be synthesizable.
- Sequential UDPs and simulation controls such as `fork...join`, `wait`, and `disable` are generally not portable hardware descriptions.

4 Summary

Commonly synthesizable constructs include module definitions and instances, parameters, gate primitives, continuous assignments, combinational `always` blocks, functions, and bounded loops. Complete `if` and `case` branches help ensure that combinational outputs are always assigned. A task may be synthesizable when it contains only supported, timing-free logic.

Constructs such as `initial`, simulation delays, `time`, real number operations, `fork...join`, and `force...release` are generally associated with testbenches or simulation rather than portable hardware. Exact support varies: for example, some FPGA flows support limited initialization, while other tools do not. Case equality, wildcard cases, and other operators also have tool- and context-dependent synthesis support; `casex` in particular can hide unknown values during simulation. Consult the synthesis tool and target-device documentation for the supported subset.